

Quantum Threats to RSA:
Shor, Grover, and Post-Quantum ML-KEM

Harry Do
Computer Science Student, Binghamton University

Technical Project Report
Spring 2026

Quantum Threats to RSA: Shor, Grover, and Post-Quantum ML-KEM

Project Summary

This project investigates how quantum computing affects RSA encryption and how post-quantum cryptography can respond.

The project follows this workflow:

RSA → Classical Factoring → Shor's Algorithm → Grover Search → ML-KEM

Main results:

- Implemented toy RSA with $n = 21$.
- Demonstrated classical factoring attack on RSA.
- Used Shor's algorithm to recover factors 3 and 7 from $N = 21$.
- Used Grover's algorithm to recover the toy RSA plaintext state 00101.
- Demonstrated ML-KEM key establishment using ML-KEM-512, ML-KEM-768, and ML-KEM-1024.

Abstract

The project investigates how quantum algorithms affect RSA encryption and how post-quantum cryptography can respond. RSA is a public-key cryptosystem where two secret prime numbers, p and q , are multiplied to create a public modulus $n = p \times q$. The security of RSA depends on the difficulty of factoring n back into p and q . In this project, I first implemented a small version of an RSA system using $p = 3$, $q = 7$, and $n = 21$. The public key was $(e, n) = (5, 21)$, the private key was $(d, n) = (5, 21)$, and the message $m = 5$ was encrypted to ciphertext $c = 17$.

After building the RSA baseline, I demonstrated a classical factoring attack showing that once $n = 21$ is factored into 3×7 , the private key can be reconstructed and the ciphertext can be decrypted. Then, I implemented Shor's algorithm for $N = 21$, where the order-finding step recovered $r = 6$, allowing the factors 3 and 7 to be recovered. I also implemented Grover's algorithm as a quantum search demonstration to recover the toy RSA plaintext from the ciphertext. Finally, I implemented ML-KEM as a post-quantum lattice-based key encapsulation method and compared ML-KEM-512, ML-KEM-768, and ML-KEM-1024. The results show that Shor's algorithm is the direct quantum threat to RSA, whereas Grover demonstrates quantum search amplification, and ML-KEM provides a post-quantum alternative for key establishment.

1. Introduction

RSA is a widely known public-key cryptosystem whose security depends on the difficulty of factoring a large integer modulus. In RSA, two prime numbers p and q are multiplied to form $n = p \times q$. The public key contains n , but the factors p and q are kept hidden. If an attacker can factor n , then the attacker can compute Euler's totient function $\phi(n) = (p - 1)(q - 1)$ and recover the private exponent d , where $d \equiv e^{-1} \pmod{\phi(n)}$. This makes factoring the central weakness of RSA.

Quantum computing changes this security assumption. Shor's algorithm can factor integers efficiently on a sufficiently powerful quantum computer, which makes it a direct threat to RSA. Grover's algorithm is different because it does not factor RSA directly, but it provides a quadratic speedup for search problems. In this project, I connect Grover to RSA by using it to search for a plaintext that matches the RSA ciphertext. The final part of the project studies ML-KEM, a post-quantum key encapsulation mechanism. Different from RSA, ML-KEM does not rely on factoring. Instead, it is based on lattice-related problems and is designed for post-quantum key establishment. The goal of this project is to connect classical RSA, quantum attacks, and post-quantum defense in one workflow.

RSA $\rightarrow$ Classical Factoring $\rightarrow$ Shor $\rightarrow$ Grover $\rightarrow$ ML-KEM

2. RSA Baseline and Classical Factoring Attack

The project begins with a toy RSA implementation using $p = 3$, $q = 7$, and $n = 21$. Euler's totient is $\phi(n) = (3 - 1)(7 - 1) = 12$. I chose the public exponent $e = 5$, which gives the private exponent $d = 5$ because $d = 5^{-1} \pmod{12}$ and $5 \times 5 = 25 \equiv 1 \pmod{12}$. Using the public key $(e, n) = (5, 21)$, I encrypted the message $m = 5$. The ciphertext follows the RSA rule $c = m^e \pmod{n}$, so $c = 5^5 \pmod{21} = 17$. Decryption uses the private key: $m = c^d \pmod{n} = 17^5 \pmod{21} = 5$. This confirms that the RSA implementation works correctly.

The classical factoring attack demonstrates why factoring breaks RSA. The attacker starts from the public modulus $n = 21$ and factors it into $21 = 3 \times 7$. Once $p = 3$ and $q = 7$ are known, the attacker computes $\phi(n) = 12$, reconstructs $d = 5$, and decrypts the ciphertext 17 back to message 5. Although $n = 21$ is only a toy value, this experiment shows the logic behind RSA's vulnerability: if n can be factored, the private key can be recovered.

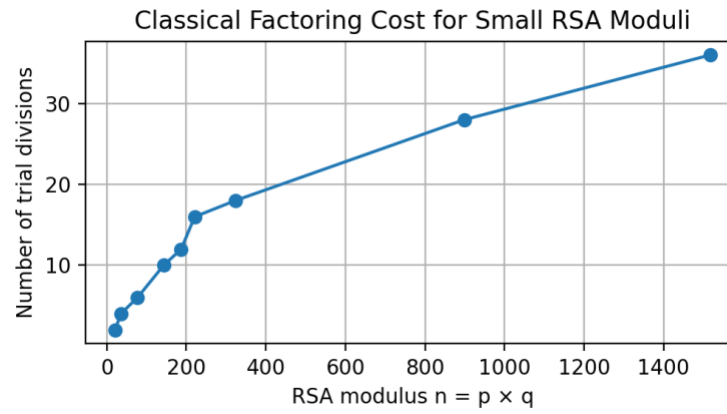


Figure 1. Classical trial division cost for small RSA-like moduli.

3. Shor's Algorithm for $N = 21$

Shor's algorithm is the direct quantum threat to RSA because it reduces factoring to an order-finding problem. For this project, I used $N = 21$ and $a = 2$. The order r is the smallest positive integer such that $2^r \equiv 1 \pmod{21}$. The modular powers are $2^1 \pmod{21} = 2$, $2^2 \pmod{21} = 4$, $2^3 \pmod{21} = 8$, $2^4 \pmod{21} = 16$, $2^5 \pmod{21} = 11$, and $2^6 \pmod{21} = 1$. Therefore, the order is $r = 6$.

Once $r = 6$ is known, the factors are recovered by classical post-processing. Since $r/2 = 3$, the algorithm computes $\gcd(2^3 - 1, 21) = \gcd(7, 21) = 7$ and $\gcd(2^3 + 1, 21) = \gcd(9, 21) = 3$. Therefore, Shor's algorithm recovers $21 = 3 \times 7$, matching the same factors used in the RSA baseline.

The important implementation change was extending the common tutorial example from $N = 15$ to $N = 21$. Many basic Shor examples use hardcoded modular multiplication gates for modulo 15. For $N = 21$, I implemented a general modular multiplication gate $|x\rangle \rightarrow |b \cdot x \pmod{21}\rangle$. Here, x is the value currently stored in the target register, and b is the modular multiplier from $b_list = [2, 4, 16, 4, 16, \dots]$. The b_list values come from powers $2^{2^k} \pmod{21}$. They do not directly give the order; instead, they build the modular exponentiation circuit, and the inverse QFT helps recover the period $r = 6$.

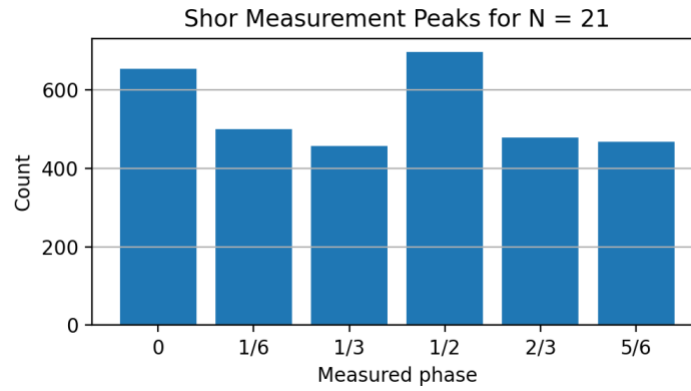


Figure 2. Shor measurement peaks for $N = 21$, consistent with order $r = 6$.

4. Grover's Algorithm for Toy RSA Plaintext Search

Grover's algorithm is different from Shor's algorithm. Shor factors the RSA modulus directly, while Grover performs quantum search. To connect Grover to RSA, I defined the search problem using public key $(e, n) = (5, 21)$ and ciphertext $c = 17$. The goal is to find a plaintext m such that $m^5 \bmod 21 = 17$. The correct plaintext is $m = 5$, which is 00101 in 5-bit binary.

Because $n = 21$, four qubits are not enough because $2^4 = 16$. Five qubits provide $2^5 = 32$ states, which can represent all values needed for this toy modulus. The Grover oracle marks states that satisfy $m^5 \bmod 21 = 17$. In this toy implementation, the satisfying state 00101 was precomputed classically and used to build the oracle. This demonstrates amplitude amplification, but it is not a fully reversible RSA-verification oracle. The simulator result recovered the state 00101, corresponding to $m = 5$, with a success probability of about 0.9992.

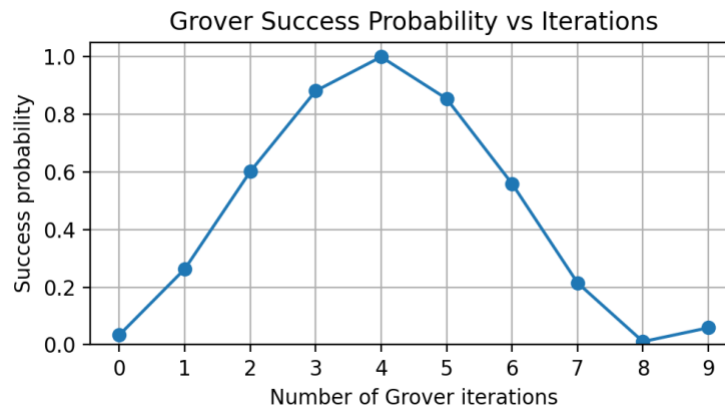


Figure 3. Grover success probability peaks around 4 iterations.

5. Post-Quantum Defense with ML-KEM

After demonstrating the quantum threat to RSA, I implemented ML-KEM as a post-quantum defense. ML-KEM is not direct message encryption like RSA. It is a key encapsulation mechanism, which means it helps two parties establish the same shared secret key. Alice generates a public/private key pair, Bob uses Alice's public key to encapsulate a shared secret, Bob sends the ciphertext to Alice, and Alice decapsulates with her private key. At the end, both Alice and Bob should obtain the same shared secret.

In the ML-KEM-512 demo, the public key length was 800 bytes, the ciphertext length was 768 bytes, and the shared secret length was 32 bytes. The result Shared secrets match: True confirms that the key establishment worked. I also compared ML-KEM-512, ML-KEM-768, and ML-KEM-1024. ML-KEM-768 used an 1184-byte public key and a 1088-byte ciphertext, while ML-KEM-1024 used a 1568-byte public key and a 1568-byte ciphertext. All parameter sets successfully established matching shared secrets.

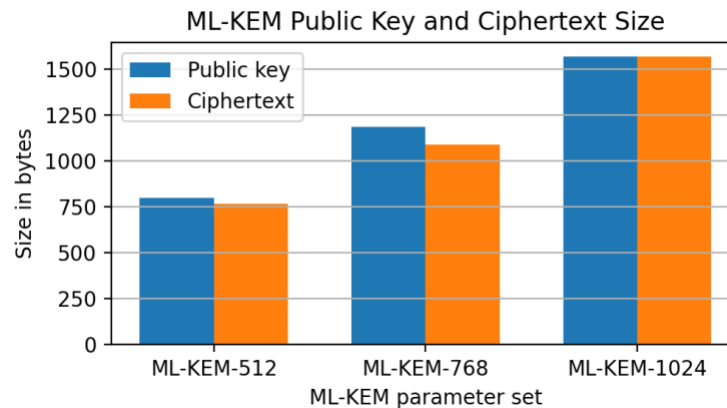


Figure 4. ML-KEM public key and ciphertext size comparison.

6. Discussion, Limitations, and Future Work

The experiments form a complete workflow. RSA provides the classical baseline, classical factoring shows why RSA depends on keeping p and q secret, Shor demonstrates the direct quantum factoring threat, Grover demonstrates quantum search amplification, and ML-KEM shows a post-quantum key-establishment method that avoids factoring.

There are several limitations. First, the RSA, Shor, and Grover examples use toy-scale numbers. The modulus $n = 21$ is intentionally small so that the math and quantum circuits remain understandable. Second, Shor's algorithm for $N = 21$ is educational and does not represent breaking real RSA. Current quantum hardware cannot run practical Shor attacks on real RSA keys. Third, the Grover oracle is simplified because the marked state was precomputed classically. A

full version would require a reversible circuit that checks $m^5 \bmod 21 = 17$ directly. Fourth, ML-KEM was implemented using a library rather than building the full lattice arithmetic from scratch.

Future work could include testing larger Shor examples, improving hardware noise analysis, implementing a full reversible Grover oracle, comparing more post-quantum schemes, and building a hybrid workflow where ML-KEM establishes a shared secret that is later used with symmetric encryption.

7. Conclusion

This project demonstrates how quantum computing changes cryptographic security assumptions. RSA depends on factoring being hard. The classical factoring experiment shows how factoring n allows private key recovery. Shor's algorithm shows the direct quantum threat by recovering the factors of $N = 21$ through order finding. Grover's algorithm demonstrates quantum search amplification by recovering the toy RSA plaintext state 00101. Finally, ML-KEM demonstrates a post-quantum key-establishment method that avoids factoring and uses lattice-based security assumptions. Overall, the project connects classical cryptography, quantum algorithms, and post-quantum defense in one coherent workflow.

Reference

- [1] IBM Quantum. “Shor’s Algorithm.” IBM Quantum Documentation.
<https://quantum.cloud.ibm.com/docs/en/tutorials/shors-algorithm>

- [2] IBM Quantum. “Grover’s Algorithm.” IBM Quantum Learning.
<https://quantum.cloud.ibm.com/learning/modules/computer-science/grovers>
- [3] National Institute of Standards and Technology (NIST). “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard.” 2024.
<https://csrc.nist.gov/pubs/fips/203/final>
- [4] Open Quantum Safe. “liboqs-python: Python 3 bindings for liboqs.” GitHub. <https://github.com/open-quantum-safe/liboqs-python>